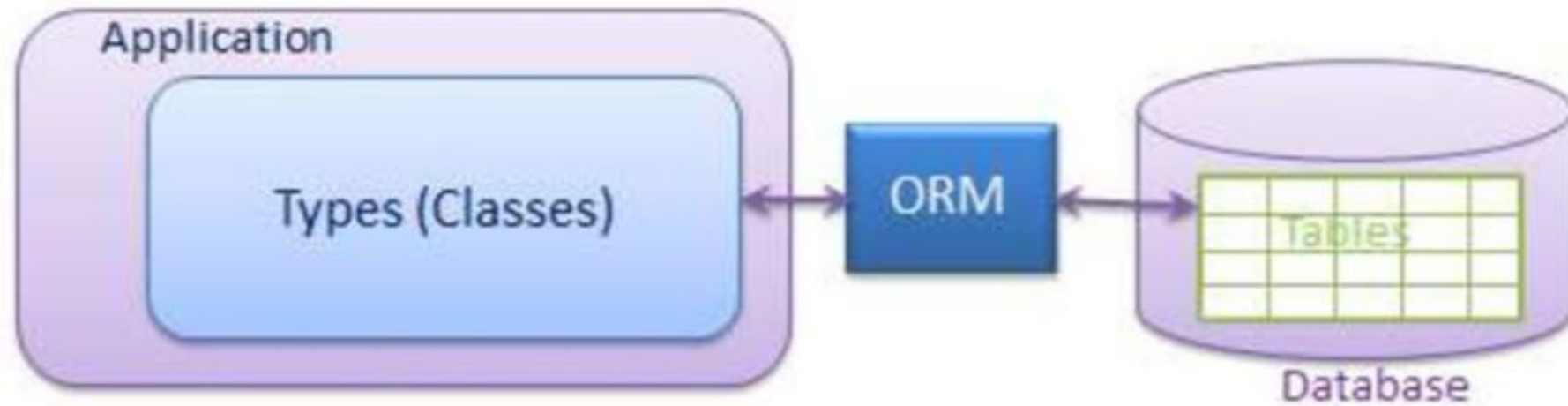


Spring Data JPA

ORM



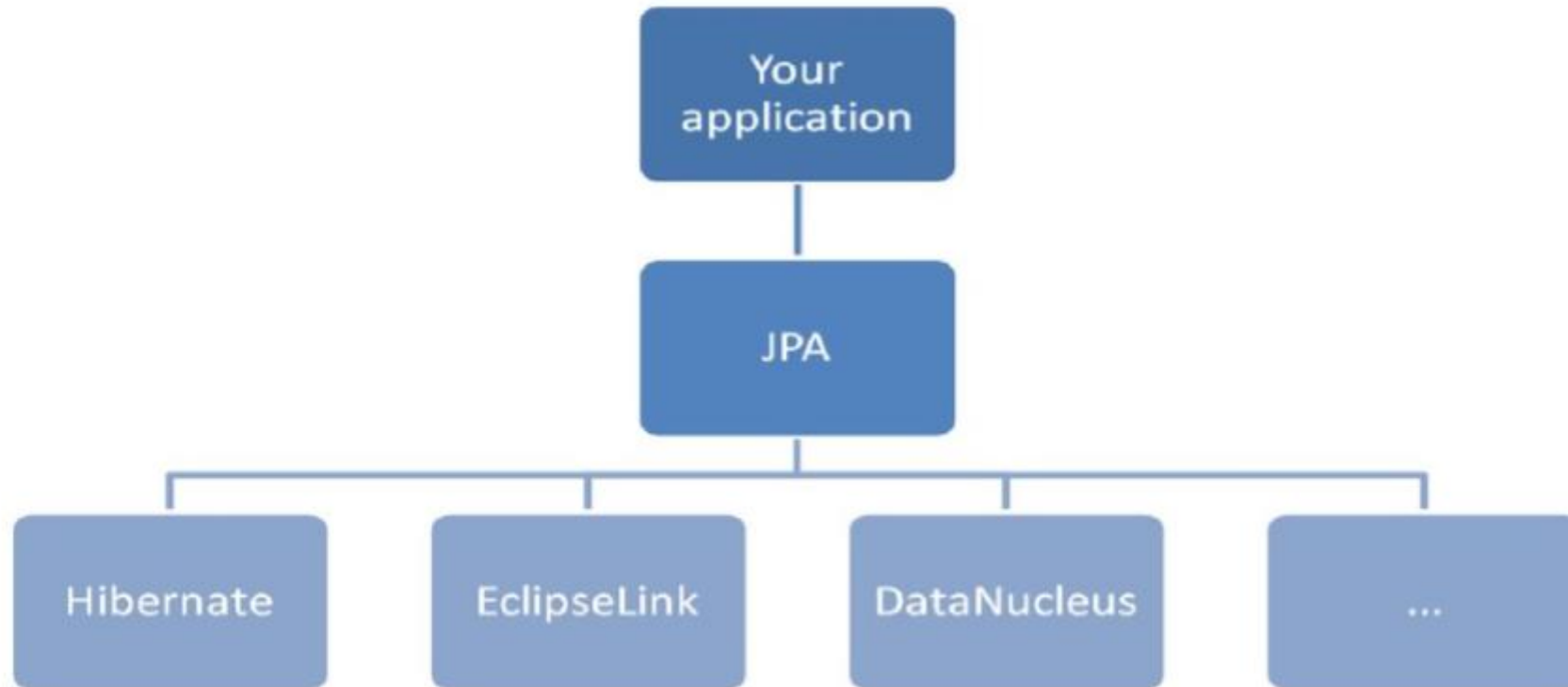
ORM(Object-Relational Mapping, рус. объектно-реляционное отображение) - технология программирования, которая позволяет обеспечить работу с данными в терминах классов, а не таблиц данных и напротив, преобразовать термины и данные классов в данные, пригодные для хранения в СУБД

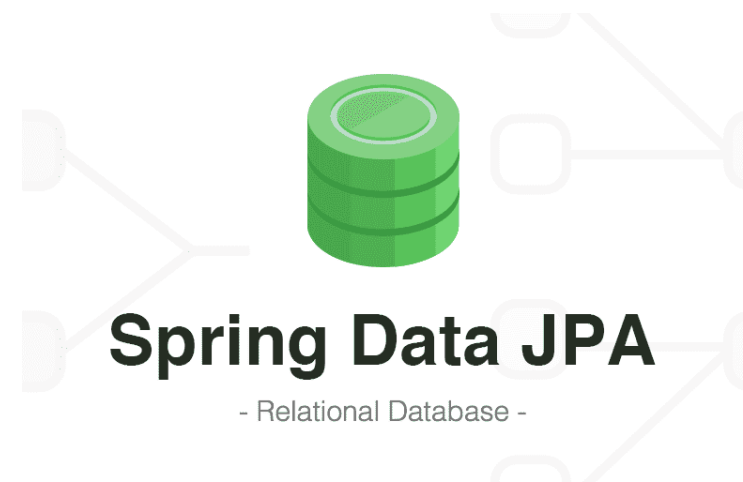
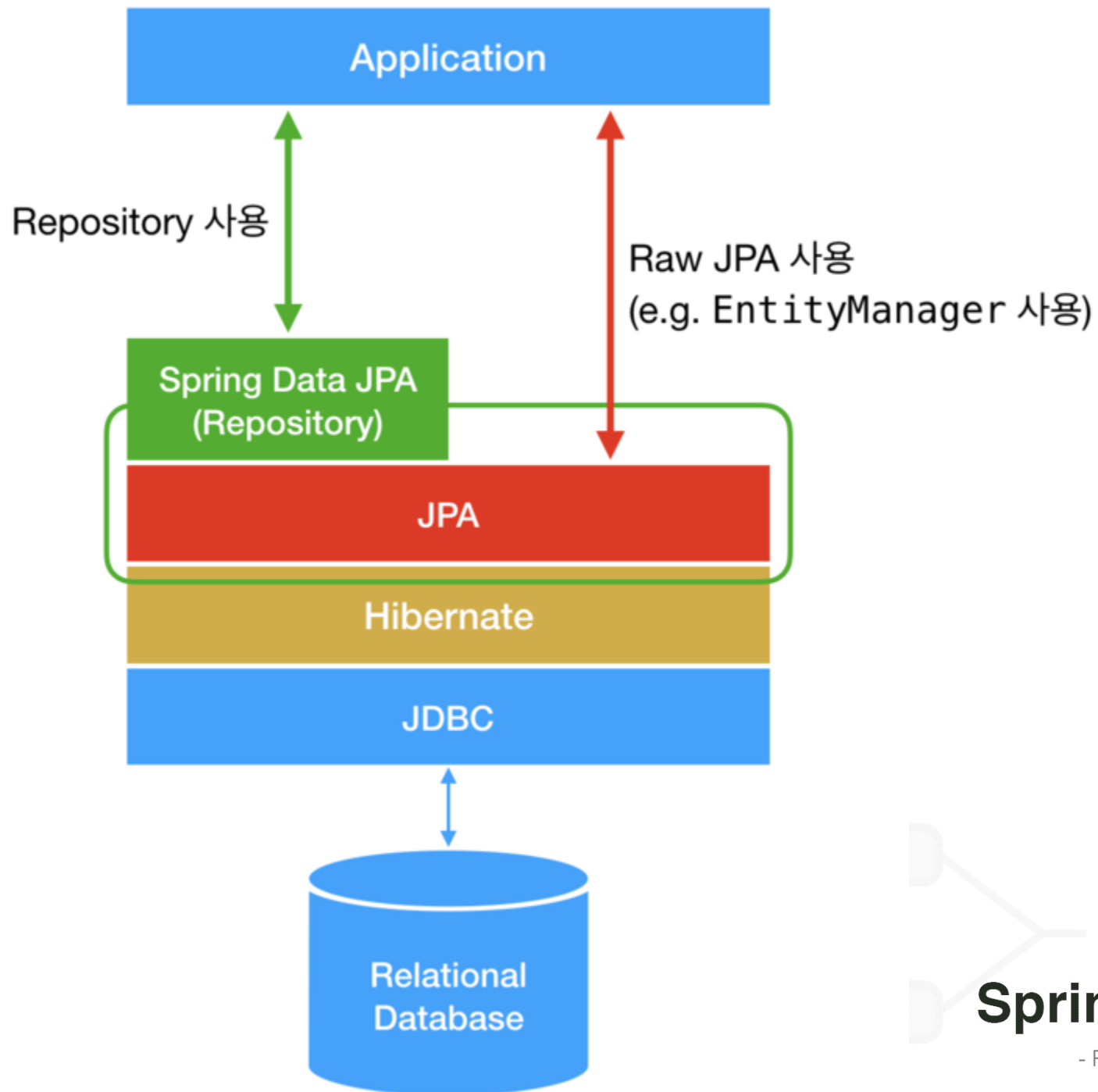


JPA



Java Persistence API (JPA) — спецификация API Java EE, предоставляет возможность сохранять в удобном виде Java-объекты в базе данных. Существует несколько реализаций этого интерфейса, одна из самых популярных использует для этого Hibernate. JPA реализует концепцию ORM.

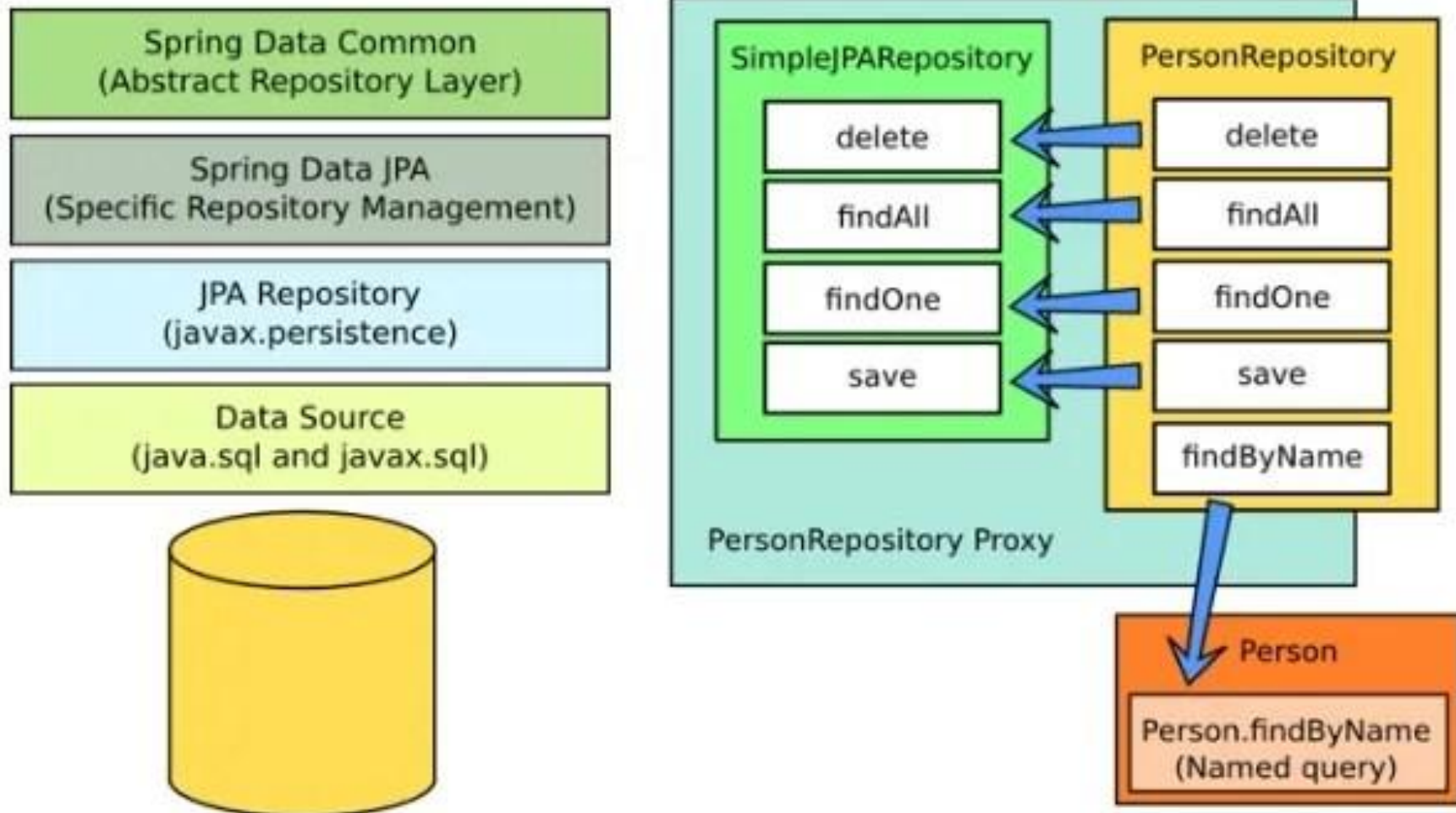




Spring Data

Дополнительный удобный механизм для взаимодействия с сущностями базы данных, организации их в репозитории, извлечение данных, изменение, в каких то случаях для этого будет достаточно объявить интерфейс и метод в нем, **без имплементации.**

Spring Data



Spring Repository

- Основное понятие в Spring Data — это репозиторий. Это несколько интерфейсов которые используют JPA Entity для взаимодействия с ней. Так например интерфейс

```
public interface CrudRepository<T, ID extends Serializable>  
    extends Repository<T, ID>
```

- обеспечивает основные операции по поиску, сохранения, удалению данных (CRUD операции)

Spring Repository

Есть и другие абстракции, например **PagingAndSortingRepository**.

Т.е. если того перечня что предоставляет интерфейс достаточно для взаимодействия с сущностью, то можно прямо расширить базовый интерфейс для своей сущности, дополнить его своими методами запросов и выполнять операции. Сейчас я покажу коротко те шаги что нужны для самого простого случая (не отвлекаясь пока на конфигурации, ORM, базу данных).

Spring Data

Традиционно классы "сущностей" JPA задаются в файле **persistence.xml**.

В Spring Boot этот файл не требуется, вместо него используется сканирование сущностей (Entity Scanning). По умолчанию поиск выполняется во всех пакетах, расположенных ниже основного конфигурационного класса (того, который аннотирован `@EnableAutoConfiguration` или `@SpringBootApplication`).

1. Создаем сущность

```
@Entity
@Table(name = "EMPLOYEES")
public class Employees {
    private Long employeeId;
    private String firstName;
    private String lastName;
    private String email;
    // . . .
}
```

2. Наследоваться от одного из интерфейсов Spring Data, например от CrudRepository

```
@Repository
```

```
public interface CustomizedEmployeesCrudRepository extends CrudRepository<Employees, Long>
```

3. Использовать в клиенте (сервисе) новый интерфейс для операций с данными

```
@Service
public class EmployeesDataService {

    @Autowired
    private CustomizedEmployeesCrudRepository employeesCrudRepository;

    @Transactional
    public void testEmployeesCrudRepository() {
        Optional<Employees> employeesOptional = employeesCrudRepository.findById(127L);
        //....
    }
}
```

3. Использовать в клиенте (сервисе) новый интерфейс для операций с данными

```
@Service
public class EmployeesDataService {

    @Autowired
    private CustomizedEmployeesCrudRepository employeesCrudRepository;

    @Transactional
    public void testEmployeesCrudRepository() {
        Optional<Employees> employeesOptional = employeesCrudRepository.findById(127L);
        //....
    }
}
```

А работает ли это?

Загадочный CrudRepository

@NoRepositoryBean

public interface CrudRepository<T, ID **extends** Serializable> **extends** Repository<T, ID> {

<S **extends** T> S save(S entity);

<S **extends** T> Iterable<S> save(Iterable<S> entities);

T findOne(ID id);

boolean exists(ID id);

Iterable<T> findAll();

Iterable<T> findAll(Iterable<ID> ids);

long count();

void delete(ID id);

void delete(T entity);

void delete(Iterable<? **extends** T> entities);

void deleteAll();

}

CrudRepository

C - Create

R - Read

U - Update

D - Delete

В следующем примере показано типичное определение интерфейса взаимодействия с репозиторием Spring Data:

Java

```
1  import org.springframework.boot.docs.data.sql.jpaaandspringdata.entityclasses.City;
2  import org.springframework.data.domain.Page;
3  import org.springframework.data.domain.Pageable;
4  import org.springframework.data.repository.Repository;
5  public interface CityRepository extends Repository<City, Long> {
6      Page<City> findAll(Pageable pageable);
7      City findByNameAndStateAllIgnoringCase(String name, String state);
8  }
```


А какие нейминг конвенции существуют?

- <https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html#jpa.query-methods.query-creation>

Keyword	Sample	JPQL snippet
Distinct	<code>findDistinctByLastnameAndFirstname</code>	<code>select distinct ... where x.lastname = ?1 and x.firstname = ?2</code>
And	<code>findByLastnameAndFirstname</code>	<code>... where x.lastname = ?1 and x.firstname = ?2</code>
Or	<code>findByLastnameOrFirstname</code>	<code>... where x.lastname = ?1 or x.firstname = ?2</code>
Is, Equals	<code>findByFirstname</code> , <code>findByFirstnameIs</code> , <code>findByFirstnameEquals</code>	<code>... where x.firstname = ?1</code>

Between	findByStartDateBetween	... where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	... where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	... where x.age <= ?1
GreaterThan	findByAgeGreaterThan	... where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	... where x.age >= ?1
After	findByStartDateAfter	... where x.startDate > ?1
Before	findByStartDateBefore	... where x.startDate < ?1
IsNull, Null	findByAge(Is)Null	... where x.age is null
IsNotNull, NotNull	findByAge(Is)NotNull	... where x.age not null
Like	findByFirstnameLike	... where x.firstname like ?1
NotLike	findByFirstnameNotLike	... where x.firstname not like ?1

StartingWith	findByFirstnameStartingWith	... where x.firstname like ?1 (parameter bound with appended %)
EndingWith	findByFirstnameEndingWith	... where x.firstname like ?1 (parameter bound with prepended %)
Containing	findByFirstnameContaining	... where x.firstname like ?1 (parameter bound wrapped in %)
OrderBy	findByAgeOrderByLastnameDesc	... where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot	... where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	... where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	... where x.age not in ?1
True	findByActiveTrue()	... where x.active = true
False	findByActiveFalse()	... where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	... where UPPER(x.firstname) = UPPER(?1)

```
internal class SubjectCourseStudentRepository : ISubjectCourseStudentRepository
{
    private readonly IPostgresConnectionProvider _connectionProvider;
    private readonly IUnitOfWork _unitOfWork;

    public void AddOrUpdateRange(IReadOnlyCollection<SubjectCourseStudent> students)
    {
        const string sql = """
            insert into subject_course_students(student_id, subject_course_id,
subject_course_student_ordinal)
            select * from unnest(:student_ids, :subject_course_ids, :ordinals)
            on conflict on constraint subject_course_students_pkey
            do update
            set subject_course_student_ordinal = excluded.subject_course_student_ordinal;
            """;

        NpgsqlCommand command = new NpgsqlCommand(sql)
            .AddParameter("student_ids", students.Select(x => x.StudentId).ToArray())
            .AddParameter("subject_course_ids", students.Select(x =>
x.SubjectCourseId).ToArray())
            .AddParameter("ordinals", students.Select(x => x.Ordinal).ToArray());

        _unitOfWork.Enqueue(command);
    }
}
```

Using @Query

Using named queries to declare queries for entities is a valid approach and works fine for a small number of queries. As the queries themselves are tied to the Java method that runs them, you can actually bind them directly by using the Spring Data JPA `@Query` annotation rather than annotating them to the domain class. This frees the domain class from persistence specific information and co-locates the query to the repository interface.

Queries annotated to the query method take precedence over queries defined using `@NamedQuery` or named queries declared in `orm.xml`.

The following example shows a query created with the `@Query` annotation:

Example 5. Declare query at the query method using @Query

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @Query("select u from User u where u.emailAddress = ?1")  
    User findByEmailAddress(String emailAddress);  
}
```

JAVA

Хочу явно реализовывать репозитории

Тоже можно.

По умолчанию, Spring Data будет «генерировать» реализацию только тех методов которые не переопределены в классах с постфиксом **Impl** (!)

@Overriding Repository

```
public class CustomizedEmployeesImpl implements CustomizedEmployees {

    @PersistenceContext
    private EntityManager em;

    @Override
    public List getEmployeesMaxSalary() {
        return em
            .createQuery(
                "from Employees where salary = (select max(salary) from Employees )",
                Employees.class)
            .getResultList();
    }
}
```

А есть ещё готовые руководства?

Конечно

<https://www.baeldung.com/spring-data>

<https://www.baeldung.com/persistence-with-spring-series>

Application properties

```
spring.datasource.url=  
    jdbc:h2:mem:db;DB_CLOSE_DELAY=-1  
spring.datasource.username=sa  
spring.datasource.password=sa
```

Включить Spring Data!

```
@Configuration  
@EnableJpaRepositories  
(basePackages =  
"org.example.data")  
class JpaConfig {}
```

А что делать с DAO слоем?

Так как Spring Data JPA менеджит большую часть рутины за вас:

- Генерирует SQL запросы
- Производит маппинг
- Управляет транзакциями (JTA)

... etc ...

То старые явные реализации как будто бы и не нужны...